



University of Engineering and Technology, Lahore

Department of Electrical Engineering

DSA LAB : 01

Name	Umama Ume Amen
Registration No.	2025-AI-264
Section	A (Afternoon)
Instructor	Ma'am Husna

Experiment 1

Classes, Constructors and Pointers in C++

OBJECTIVE:

- Define and use classes in C++
- Implement constructors
- Create objects on stack and heap
- Understand pointer fundamentals
- Use dynamic memory allocation (new and delete)
- Use pointer to object and arrow operator (->)

INTRODUCTION:

Before studying advanced data structures such as Linked Lists, Trees, and Graphs, it is essential to understand how classes, objects, pointers, and dynamic memory work in C++. This lab provides the foundation required for implementing dynamic data structures.

INTRODUCTION TO CLASSES

A class in C++ is a user-defined data type that acts as a blueprint for creating objects. It allows us to combine data and functions into a single unit.

- A class defines what an object contains (data members).
- A class defines what an object can do (member functions).
- An object is an instance of a class.

BASIC STRUCTURE OF A CLASS:

```
class ClassName {  
  
public:  
  
    // data members  
  
    // member functions  
  
};
```

Important Concepts:

- **Data Members:** Variables defined inside the class.
- **Member Functions:** Functions that operate on the data members.

- **Access Specifiers:**
 - o public → Accessible from outside the class.
 - o private → Accessible only inside the class.
- The . operator is used to access members of an object.

TASK 1 – CREATE AND USE A SIMPLE CLASS

Write the following program and execute it:

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollNo;

    void display() {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
    }
};

int main() {
    Student s1;
    s1.name = "Ali";
    s1.rollNo = 101;
    s1.display();
    return 0;
}
```

(1)

CONSTRUCTORS AND OBJECT CREATION

A constructor has the same name as the class, has no return type, and initializes object data automatically.

TASK 2 – ADD A CONSTRUCTOR

Modify the class as follows:

```
class Student {
public:
    string name;
    int rollNo;

    Student(string n, int r) {
```

```
        name = n;
        rollNo = r;
    }
    void display() {
        cout << name << " - " << rollNo << endl;
    }
};

int main() {
    Student s1("Sara", 102);
    s1.display();
    return 0;
}
```

(2)

POINTER FUNDAMENTALS

A pointer:

- Stores the address of a variable.
- Uses * for declaration.
- Uses & to get address.
- Uses * to access value (dereference).

TASK 3: BASIC POINTER EXAMPLE

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int* ptr = &x;

    cout << "Value of x: " << x << endl;
    cout << "Address of x: " << &x << endl;
    cout << "Pointer value: " << ptr << endl;
    cout << "Value via pointer: " << *ptr << endl;

    return 0;
}
```

Observation

- ptr stores address of x
- *ptr gives value stored at that address

DYNAMIC MEMORY (NEW AND DELETE)

In C++, memory can be allocated in two main areas:

1. Stack Memory

- Automatically managed by the compiler.
- Used for local variables and function calls.
- Memory is released automatically when the function ends.
- Fast but limited in size.

Example:

```
int x = 10;
```

Here, memory for x is created in the stack and automatically destroyed when the program leaves its scope.

2. Heap Memory (Dynamic Memory)

- Allocated manually during runtime.
- Memory remains allocated until it is explicitly released.
- Used when we need flexible or large memory allocation.
- Essential for dynamic data structures like Linked Lists, Trees, and Graphs.

To allocate memory in heap, we use the **new** operator:

```
int* p = new int;
```

This statement:

- Allocates memory in heap.
- Returns the address of that memory.
- Stores the address in pointer p.

To release the memory, we use the **delete operator**:

```
delete p;
```

Task 4: Dynamic Integer:

```
#include <iostream>
using namespace std;

int main() {
    int* p = new int;

    *p = 25;
```

```
cout << "Value: " << *p << endl;

delete p;

return 0;
}
```

(4)

POINTER TO OBJECT & ARROW OPERATOR

There are two ways to create objects:

1. STACK OBJECT

```
Student s1("Ali", 101);
```

```
//Access using:
```

```
s1.display();
```

2. HEAP OBJECT

```
Student* s2 = new Student("Ahmed", 105);
```

```
//Access using:
```

```
s2->display();
```

```
// -> is used to access members via pointer.
```

TASK 5: STACK VS HEAP OBJECT

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollNo;

    Student(string n, int r) {
        name = n;
        rollNo = r;
    }
}
```

```
void display() {  
    cout << name << " - " << rollNo << endl;  
}  
};  
  
int main() {  
  
    Student s1("Ali", 101);  
    s1.display();  
  
    Student* s2 = new Student("Ahmed", 105);  
    s2->display();  
  
    delete s2;  
  
    return 0;  
}
```

(5)

TASK 6: CREATE DYNAMIC STUDENT RECORDS

```
#include <iostream>  
using namespace std;  
  
class Student {  
public:  
    string name;  
    int rollNo;  
  
    Student() {  
        name = "";  
        rollNo = 0;  
    }  
  
    void setData(string n, int r) {  
        name = n;  
        rollNo = r;  
    }  
  
    void display() {  
        cout << name << " - " << rollNo << endl;  
    }  
};
```

```
    }  
};  
  
int main() {  
  
    int n;  
    cout << "Enter number of students: ";  
    cin >> n;  
  
    Student* students = new Student[n]; // dynamic array of objects  
  
    for(int i = 0; i < n; i++) {  
        string name;  
        int roll;  
  
        cout << "Enter name and roll: ";  
        cin >> name >> roll;  
  
        students[i].setData(name, roll);  
    }  
  
    cout << "\nStudent List:\n";  
    for(int i = 0; i < n; i++) {  
        students[i].display();  
    }  
  
    delete[] students; // very important  
  
    return 0;  
}
```

(6)

LAB EXERCISE:

1. Create a Car class with brand and model, constructor, and display function.
2. Create a dynamic integer array of size 5 using new. Take input, print values, and free memory.
3. Write a program that:
 - a. Dynamically creates two Car objects using new
 - b. Stores them using pointers
 - c. Displays their details using ->
 - d. Deletes both objects properly

Lab Task 1 - Student Class and Objects

Code and output

Screenshot of the completed C++ program and its execution result.

```
task1.cpp > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Student {
5  public:
6      string name;
7      int rollNo;
8      void display() {
9          cout << "Name: " << name << endl;
10         cout << "Roll No: " << rollNo << endl;
11     }
12 };
13 int main() {
14     Student s1, s2, s3;
15     s1.name = "Ali";
16     s1.rollNo = 101;
17     s1.display();
18     s2.name = "Aina";
19     s2.rollNo = 264;
20     s2.display();
21     s3.name = "Ayehsa";
22     s3.rollNo = 980;
23     s3.display();
24     return 0;
25 }
```

Problems Output Debug Console Terminal Ports

PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if (\$?) { g++ task1.cpp -o task1 } ; if (\$?) { .\task1 }

Name: Ali
Roll No: 101
Name: Aina
Roll No: 264
Name: Ayehsa
Roll No: 980

PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1>

Lab Task 2 - Constructor in Student Class

Code and output

Screenshot of the completed C++ program and its execution result.

```
task2.cpp > Student > rollNo
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Student
5  { public:
6    string name;
7    int rollNo;
8    Student(string n, int r) // constructor
9    // Constructor name must match class name: Student
10   //It has no return type, so do not write void Student(...)
11   //It runs automatically when an object is created
12   {name=n; rollNo=r; }
13   void display()
14   { cout<<name<<"-"<< rollNo<<endl; }
15   };
16   int main()
17   { Student s1("Sara",102);
18     s1.display();
19     Student s2("Alia",245);
20     s2.display();
21     Student s3("Ahmed",300);
22     s3.display();
23     return 0;
24   }
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++
Name: Ayehsa
Roll No: 980
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++
task2.cpp -o task2 } ; if ($?) { .\task2 }
Sara-102
Alia-245
Ahmed-300
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> 
```

Screenshot of the completed C++ program and its execution result.

```
task3.cpp > ...
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int main()
5  {
6      int x=1000000;
7      int*ptr=&x; //stores the address of x inside ptr.
8      cout<<"Value of x: "<<x<<endl;
9      cout<<"Address of x: "<<&x<<endl;
10     cout<<"pointer value: "<<ptr<<endl;
11     cout<<"Value via ptr: "<<*ptr<<endl;
12     return 0 ;
13 }
14
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++
task2.cpp -o task2 } ; if ($?) { .\task2 }
● PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++
task3.cpp -o task3 } ; if ($?) { .\task3 }
Value of x: 1000000
Address of x: 0x61ff08
pointer value: 0x61ff08
Value via ptr: 1000000
○ PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> 
```

Lab Task 4 - Dynamic Integer using new and delete

Code and output

Screenshot of the completed C++ program and its execution result.

```
task4.cpp > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int main()
5  {
6      int* p= new int; //new int creates one integer box in heap memory.
7                        //It gives us the address of that box.
8                        //Pointer p stores that address.
9      *p=10;
10     cout<<"Value of p: "<<*p<<endl;
11     delete p; // delete that initialized address
12     return 0;
13 }
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++ task3.cpp -o task3 } ; if ($?) { .\task3 }
Address of x: 0x611108
pointer value: 0x61ff08
Value via ptr: 1000000
● PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++ task4.cpp -o task4 } ; if ($?) { .\task4 }
Value of p: 10
○ PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> 
```

Lab Task 5 - Stack and Heap Objects

Code and output

Screenshot of the completed C++ program and its execution result.

```
task5.cpp > Student > display()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Student {
5  public :
6  string name;
7  int rollNo ;
8  Student(string n, int r)
9  {
10 name = n;
11 rollNo = r;
12 }
13 void display() {
14 cout << name << " " << rollNo << endl;
15 }
16 };
17 int main() {
18 Student s1("Ali", 101 ); // stack object
19 s1.display(); // simple normal stack display okay
20 Student* s2 = new Student("Ahmed", 105 ); // heap object
21 s2->display(); //heap use arrow stack use dot
22 delete s2; // delete heap obj
23 return 0;
24 }
```

Problems Output Debug Console Terminal Ports

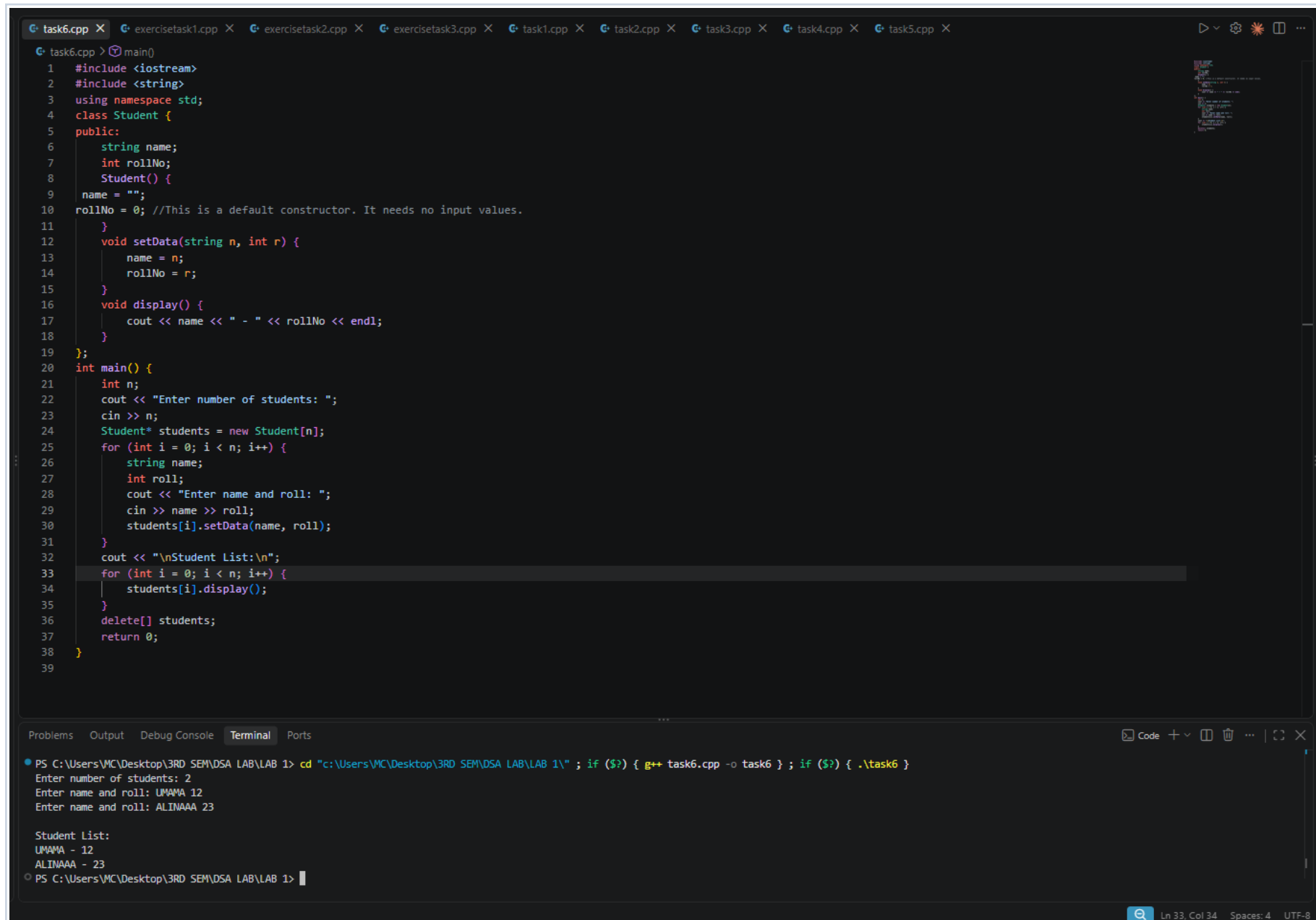
Code + - [] [X] ... [] [X]

```
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++ task5.
cpp -o task5 } ; if ($?) { .\task5 }
Ali 101
Ahmed 105
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> [ ]
```

Lab Task 6 - Dynamic Array of Student Objects

Code and output

Screenshot of the completed C++ program and its execution result.



```
task6.cpp X exercisetask1.cpp X exercisetask2.cpp X exercisetask3.cpp X task1.cpp X task2.cpp X task3.cpp X task4.cpp X task5.cpp X
task6.cpp > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Student {
5  public:
6      string name;
7      int rollNo;
8      Student() {
9          name = "";
10         rollNo = 0; //This is a default constructor. It needs no input values.
11     }
12     void setData(string n, int r) {
13         name = n;
14         rollNo = r;
15     }
16     void display() {
17         cout << name << " - " << rollNo << endl;
18     }
19 };
20 int main() {
21     int n;
22     cout << "Enter number of students: ";
23     cin >> n;
24     Student* students = new Student[n];
25     for (int i = 0; i < n; i++) {
26         string name;
27         int roll;
28         cout << "Enter name and roll: ";
29         cin >> name >> roll;
30         students[i].setData(name, roll);
31     }
32     cout << "\nStudent List:\n";
33     for (int i = 0; i < n; i++) {
34         students[i].display();
35     }
36     delete[] students;
37     return 0;
38 }
39

Problems Output Debug Console Terminal Ports
PS C:\Users\VMC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\VMC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++ task6.cpp -o task6 }; if ($?) { .\task6 }
Enter number of students: 2
Enter name and roll: UMAMA 12
Enter name and roll: ALINAAA 23

Student List:
UMAMA - 12
ALINAAA - 23
PS C:\Users\VMC\Desktop\3RD SEM\DSA LAB\LAB 1>
```

Exercise 1 - Car Objects: Stack and Heap - Program Code

Code and output

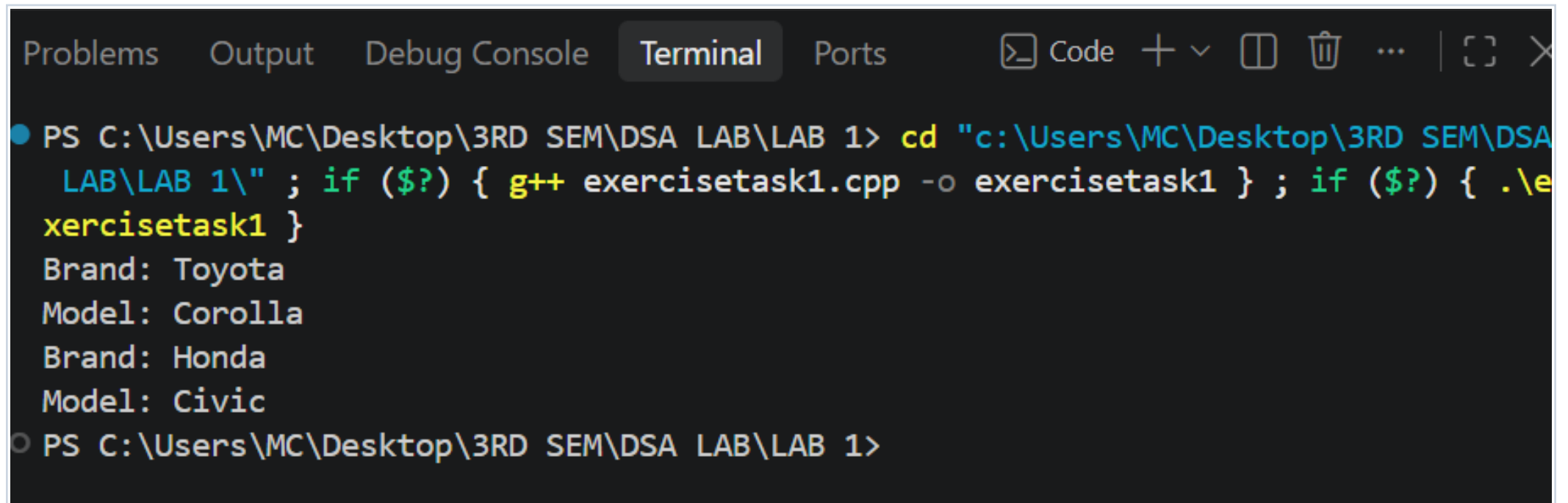
Screenshot of the completed C++ program and its execution result.

```
G+ exercisetask1.cpp > ...
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Car {
5  public:
6      string brand;
7      string model;
8      Car(string b, string m) {
9          brand = b;
10         model = m;
11     }
12     void display() {
13         cout << "Brand: " << brand << endl;
14         cout << "Model: " << model << endl;
15     }
16 };
17 int main() {
18     Car c1("Toyota", "Corolla"); //static
19     c1.display(); //call by dot
20     Car* c2 = new Car("Honda", "Civic"); //heap
21     c2->display(); // call by arrow
22     delete c2; // delete heap
23     return 0;
24 }
25
```

Exercise 1 - Car Objects: Stack and Heap - Program Output

Code and output

Screenshot of the completed C++ program and its execution result.



```
Problems  Output  Debug Console  Terminal  Ports  Code  +  -  [ ]  [X]  ...  |  [ ]  X

● PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++ exercisetask1.cpp -o exercisetask1 } ; if ($?) { .\exercisetask1 }
Brand: Toyota
Model: Corolla
Brand: Honda
Model: Civic
○ PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1>
```

Exercise 2 - Dynamic Integer Array - Program Code

Code and output

Screenshot of the completed C++ program and its execution result.

```
exercisetask2.cpp > main()
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int* arr = new int[5]; // heap
5      for (int i = 0; i < 5; i++) {
6          cout << "Enter " << i + 1 << " number : ";
7          cin >> arr[i];
8      }
9      cout << "\nArray elements are:\n";
10     for (int i = 0; i < 5; i++) {
11         cout << arr[i] << endl;
12     }
13     delete[] arr;
14     return 0;
15 }
16
```

Exercise 2 - Dynamic Integer Array - Program Output

Code and output

Screenshot of the completed C++ program and its execution result.

```
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA  
Model: Civic  
● PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA  
LAB\LAB 1\" ; if ($?) { g++ exercisetask2.cpp -o exercisetask2 } ; if ($?) { .\exe  
rcisetask2 }  
Enter 1 number : 12  
Enter 2 number : 32  
Enter 3 number : 123  
Enter 4 number : 234  
Enter 5 number : 234  
  
Array elements are:  
12  
32  
123  
234  
234  
○ PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1>
```

Exercise 3 - Two Dynamic Car Objects - Program Code

Code and output

Screenshot of the completed C++ program and its execution result.

```
exercisetask3.cpp > Car > display()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Car {
5  public:
6      string brand;
7      string model;
8      Car(string b, string m) {
9          brand = b;
10         model = m;
11     }
12     void display() {
13         cout << "Brand: " << brand << endl;
14         cout << "Model: " << model << endl;
15     }
16 };
17 int main() {
18     // create two Cars and store them in pointers
19     Car* car1 = new Car("Toyota", "Corolla");
20     Car* car2 = new Car("Honda", "Civic");
21     // c: display details using arrow operator
22     car1->display();
23     car2->display();
24     // delete both heap objects properly
25     delete car1;
26     delete car2;
27     return 0;
28 }
```

Exercise 3 - Two Dynamic Car Objects - Program Output

Code and output

Screenshot of the completed C++ program and its execution result.

```
254
● PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1\" ; if ($?) { g++ exercisetask3.cpp -o exercisetask3 } ; if ($?) { .\exercisetask3 }
Brand: Toyota
Model: Corolla
Brand: Honda
Model: Civic
○ PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\LAB 1> █
```